Relational Model concept

Relational model can represent as a table with columns and rows. Each row is known as a tuple. Each table of the column has a name or attribute.

Domain: It contains a set of atomic values that an attribute can take.

Attribute: It contains the name of a column in a particular table. Each attribute Ai must have a domain, dom(Ai)

Relational instance: In the relational database system, the relational instance is represented by a finite set of tuples. Relation instances do not have duplicate tuples.

Relational schema: A relational schema contains the name of the relation and name of all columns or attributes.

Relational key: In the relational key, each row has one or more attributes. It can identify the row in the relation uniquely.

Example: STUDENT Relation

NAME	ROLL_NO	PHONE_NO	ADDRESS	AGE
Ram	14795	7305758992	Noida	24
Shyam	12839	9026288936	Delhi	35
Laxman	33289	8583287182	Gurugram	20
Mahesh	27857	7086819134	Ghaziabad	27
Ganesh	17282	9028 9i3988	Delhi	40

- In the given table, NAME, ROLL_NO, PHONE_NO, ADDRESS, and AGE are the attributes.
- The instance of schema STUDENT has 5 tuples.
- t3 = <Laxman, 33289, 8583287182, Gurugram, 20>

Properties of Relations

- Name of the relation is distinct from all other relations.
- Each relation cell contains exactly one atomic (single) value
- Each attribute contains a distinct name

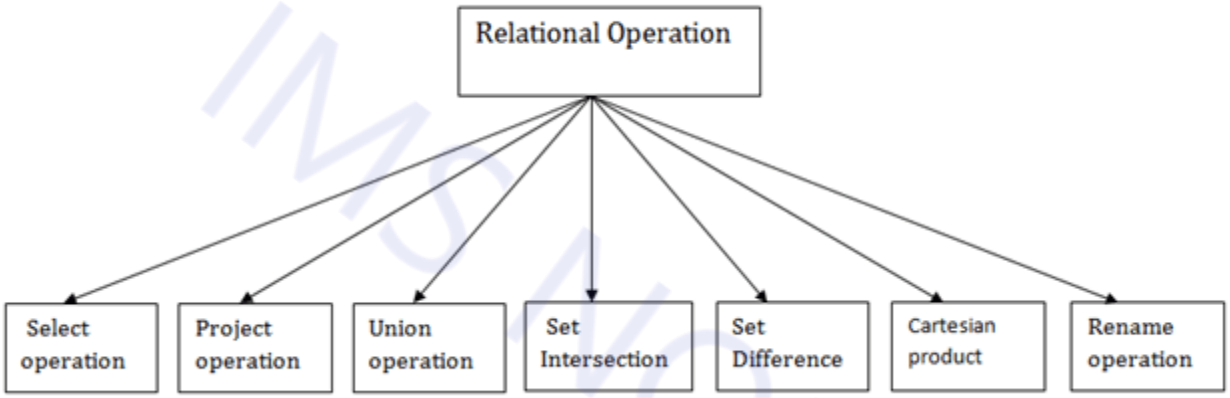
IMS NOIDA

- Attribute domain has no significance
- tuple has no duplicate value
- Order of tuple can have a different sequence

Relational Algebra

Relational algebra is a procedural query language. It gives a step by step process to obtain the result of the query. It uses operators to perform queries.

Types of Relational operation



IMS NOIDA

1. Select Operation:

- The select operation selects tuples that satisfy a given predicate.
- It is denoted by sigma (σ).

1. Notation: σ p(r)
   Where:

   σ is used for selection prediction
   r is used for relation
   p is used as a propositional logic formula which may use connectors like: AND OR and NOT. These relational can use as relational operators like =, ≠, ≥, <, >, ≤.

   For example: LOAN Relation

BRANCH_NAME	LOAN_NO	AMOUNT
Downtown	L-17	1000
Redwood	L-23	2000
Perryride	L-15	1500
Downtown	L-14	1500
Mianus	L-13	500
Roundhill	L-11	900
Perryride	L-16	1300

Input:

1. σ BRANCH_NAME="perryride" (LOAN)

Output:

BRANCH_NAME	LOAN_NO	AMOUNT
Perryride	L-15	1500
Perryride	L-16	1300

2. Project Operation:

- This operation shows the list of those attributes that we wish to appear in the result. Rest of the attributes are eliminated from the table.
- It is denoted by ∏.

1. Notation: ∏ A1, A2, An (r)

Where

A1, **A2**, **A3** is used as an attribute name of relation **r**.

IMS NOIDA

Example: CUSTOMER RELATION

NAME	STREET	CITY
Jones	Main	Harrison
Smith	North	Rye
Hays	Main	Harrison
Curry	North	Rye
Johnson	Alma	Brooklyn
Brooks	Senator	Brooklyn

Input:

1. ∏ NAME, CITY (CUSTOMER)

Output:

NAME	CITY
Jones	Harrison
Smith	Rye
Hays	Harrison
Curry	Rye
Johnson	Brooklyn
Brooks	Brooklyn

IMS NOIDA

3. Union Operation:

- Suppose there are two tuples R and S. The union operation contains all the tuples that are either in R or S or both in R & S.
- It eliminates the duplicate tuples. It is denoted by ∪.

1. Notation: R ∪ S
   A union operation must hold the following condition:
   - R and S must have the attribute of the same number.
   - Duplicate tuples are eliminated automatically.

Example:

DEPOSITOR RELATION

CUSTOMER_NAME	ACCOUNT_NO
Johnson	A-101
Smith	A-121
Mayes	A-321
Turner	A-176
Johnson	A-273
Jones	A-472
Lindsay	A-284

BORROW RELATION

CUSTOMER_NAME	LOAN_NO
Jones	L-17

IMS NOIDA

Smith	L-23
Hayes	L-15
Jackson	L-14
Curry	L-93
Smith	L-11
Williams	L-17

Input:

1. ∏ CUSTOMER_NAME (BORROW) ∪ ∏ CUSTOMER_NAME (DEPOSITOR)

Output:

CUSTOMER_NAME
Johnson
Smith
Hayes
Turner
Jones
Lindsay
Jackson
Curry

IMS NOIDA

Williams
Mayes

4. Set Intersection:

- Suppose there are two tuples R and S. The set intersection operation contains all tuples that are in both R & S.
- It is denoted by intersection ∩.

1. Notation: R ∩ S

   Example: Using the above DEPOSITOR table and BORROW table

   Input:

1. ∏ CUSTOMER_NAME (BORROW) ∩ ∏ CUSTOMER_NAME (DEPOSITOR)

   Output:

CUSTOMER_NAME
Smith
Jones

5. Set Difference:

- Suppose there are two tuples R and S. The set intersection operation contains all tuples that are in R but not in S.
- It is denoted by intersection minus (-).

1. Notation: R - S

   Example: Using the above DEPOSITOR table and BORROW table

   Input:

1. ∏ CUSTOMER_NAME (BORROW) - ∏ CUSTOMER_NAME (DEPOSITOR)

   Output:

CUSTOMER_NAME

IMS NOIDA

Jackson
Hayes
Willians
Curry

6. Cartesian product

- The Cartesian product is used to combine each row in one table with each row in the other table. It is also known as a cross product.
- It is denoted by X.

1. Notation: E X D

Example:

EMPLOYEE

EMP_ID	EMP_NAME	EMP_DEPT
1	Smith	A
2	Harry	C
3	John	B

DEPARTMENT

DEPT_NO	DEPT_NAME
A	Marketing
B	Sales
C	Legal

IMS NOIDA

Input:

1. EMPLOYEE X DEPARTMENT

Output:

EMP_ID	EMP_NAME	EMP_DEPT	DEPT_NO	DEPT_NAME
1	Smith	A	A	Marketing
1	Smith	A	B	Sales
1	Smith	A	C	Legal
2	Harry	C	A	Marketing
2	Harry	C	B	Sales
2	Harry	C	C	Legal
3	John	B	A	Marketing
3	John	B	B	Sales
3	John	B	C	Legal

IMS NOIDA

7. Rename Operation:

The rename operation is used to rename the output relation. It is denoted by **rho** (ρ).

Example: We can use the rename operator to rename STUDENT relation to STUDENT1.

1. ρ(STUDENT1, STUDENT)

> *Note: Apart from these common operations Relational algebra can be used in Join operations.*

Join Operations:

A Join operation combines related tuples from different relations, if and only if a given join condition is satisfied. It is denoted by ⋈.

Example:

EMPLOYEE

EMP_CODE	EMP_NAME
101	Stephan
102	Jack
103	Harry

SALARY

EMP_CODE	SALARY
101	50000
102	30000
103	25000

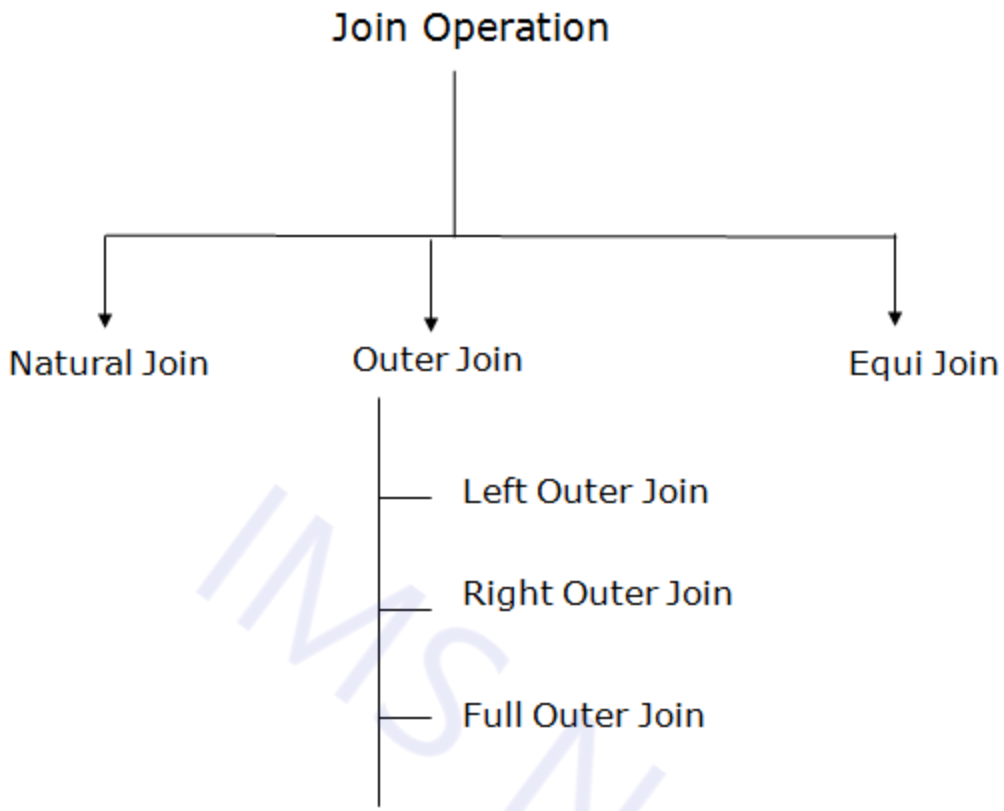
1. Operation: (EMPLOYEE ⋈ SALARY)

   Result:

EMP_CODE	EMP_NAME	SALARY
101	Stephan	50000
102	Jack	30000
103	Harry	25000

Types of Join operations:

IMS NOIDA



1. Natural Join:

- A natural join is the set of tuples of all combinations in R and S that are equal on their common attribute names.
- It is denoted by ⋈.

Example: Let's use the above EMPLOYEE table and SALARY table:

Input:

1. ∏EMP_NAME, SALARY (EMPLOYEE ⋈ SALARY)

Output:

EMP_NAME	SALARY
Stephan	50000

IMS NOIDA

Jack	30000
Harry	25000

2. Outer Join:

The outer join operation is an extension of the join operation. It is used to deal with missing information.

Example:

EMPLOYEE

EMP_NAME	STREET	CITY
Ram	Civil line	Mumbai
Shyam	Park street	Kolkata
Ravi	M.G. Street	Delhi
Hari	Nehru nagar	Hyderabad

FACT_WORKERS

EMP_NAME	BRANCH	SALARY
Ram	Infosys	10000
Shyam	Wipro	20000
Kuber	HCL	30000
Hari	TCS	50000

Input:

1. (EMPLOYEE ⋈ FACT_WORKERS)

Output:

IMS NOIDA

EMP_NAME	STREET	CITY	BRANCH	SALARY
Ram	Civil line	Mumbai	Infosys	10000
Shyam	Park street	Kolkata	Wipro	20000
Hari	Nehru nagar	Hyderabad	TCS	50000

An outer join is basically of three types:

a. Left outer join
b. Right outer join
c. Full outer join

a. Left outer join:

- Left outer join contains the set of tuples of all combinations in R and S that are equal on their common attribute names.
- In the left outer join, tuples in R have no matching tuples in S.
- It is denoted by ⟕.

Example: Using the above EMPLOYEE table and FACT_WORKERS table

Input:

1. EMPLOYEE ⟕ FACT_WORKERS

EMP_NAME	STREET	CITY	BRANCH	SALARY
Ram	Civil line	Mumbai	Infosys	10000
Shyam	Park street	Kolkata	Wipro	20000
Hari	Nehru street	Hyderabad	TCS	50000
Ravi	M.G. Street	Delhi	NULL	NULL

IMS NOIDA

b. Right outer join:

- Right outer join contains the set of tuples of all combinations in R and S that are equal on their common attribute names.
- In right outer join, tuples in S have no matching tuples in R.
- It is denoted by ⟖.

Example: Using the above EMPLOYEE table and FACT_WORKERS Relation

Input:

1. EMPLOYEE ⟖ FACT_WORKERS

Output:

EMP_NAME	BRANCH	SALARY	STREET	CITY
Ram	Infosys	10000	Civil line	Mumbai
Shyam	Wipro	20000	Park street	Kolkata
Hari	TCS	50000	Nehru street	Hyderabad
Kuber	HCL	30000	NULL	NULL

c. Full outer join:

- Full outer join is like a left or right join except that it contains all rows from both tables.
- In full outer join, tuples in R that have no matching tuples in S and tuples in S that have no matching tuples in R in their common attribute name.
- It is denoted by ⟗.

Example: Using the above EMPLOYEE table and FACT_WORKERS table

Input:

1. EMPLOYEE ⟗ FACT_WORKERS

Output:

EMP_NAME	STREET	CITY	BRANCH	SALARY

IMS NOIDA

Ram	Civil line	Mumbai	Infosys	10000
Shyam	Park street	Kolkata	Wipro	20000
Hari	Nehru street	Hyderabad	TCS	50000
Ravi	M.G. Street	Delhi	NULL	NULL
Kuber	NULL	NULL	HCL	30000

3. Equi join:

It is also known as an inner join. It is the most common join. It is based on matched data as per the equality condition. The equi join uses the comparison operator(=).

Example:

CUSTOMER RELATION

CLASS_ID	NAME
1	John
2	Harry
3	Jackson

PRODUCT

PRODUCT_ID	CITY
1	Delhi
2	Mumbai
3	Noida

Input:

IMS NOIDA

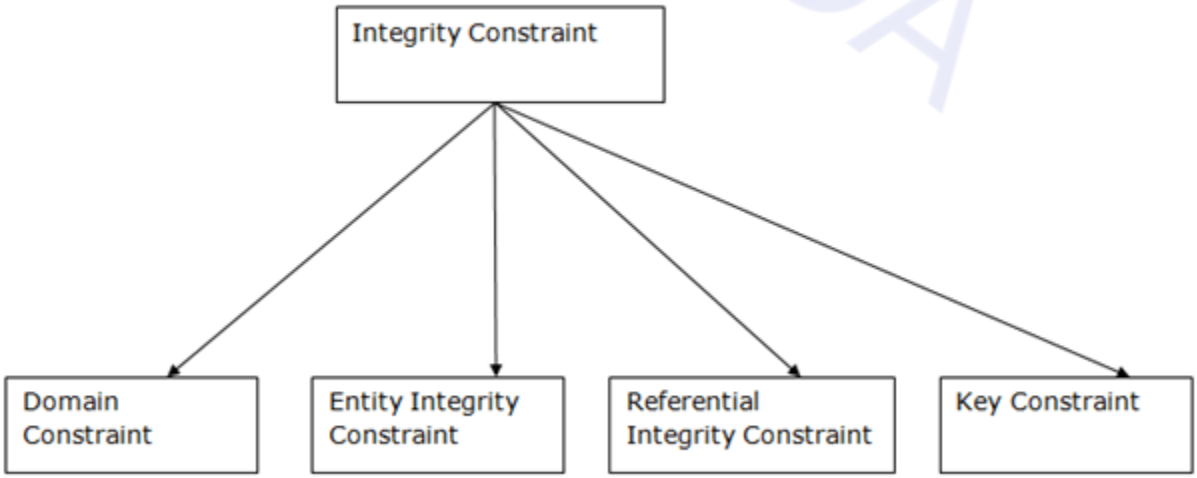
1. CUSTOMER ⋈ PRODUCT
   Output:

CLASS_ID	NAME	PRODUCT_ID	CITY
1	John	1	Delhi
2	Harry	2	Mumbai
3	Harry	3	Noida

Integrity Constraints

- Integrity constraints are a set of rules. It is used to maintain the quality of information.
- Integrity constraints ensure that the data insertion, updating, and other processes have to be performed in such a way that data integrity is not affected.
- Thus, integrity constraint is used to guard against accidental damage to the database.

Types of Integrity Constraint



IMS NOIDA

1. Domain constraints

- Domain constraints can be defined as the definition of a valid set of values for an attribute.
- The data type of domain includes string, character, integer, time, date, currency, etc. The value of the attribute must be available in the corresponding domain.

Example:

ID	NAME	SEMENSTER	AGE
1000	Tom	1st	17
1001	Johnson	2nd	24
1002	Leonardo	5th	21
1003	Kate	3rd	19
1004	Morgan	8th	A

Not allowed. Because AGE is an integer attribute

2. Entity integrity constraints

- The entity integrity constraint states that primary key value can't be null.
- This is because the primary key value is used to identify individual rows in relation and if the primary key has a null value, then we can't identify those rows.
- A table can contain a null value other than the primary key field.

Example:

EMPLOYEE

EMP_ID	EMP_NAME	SALARY
123	Jack	30000
142	Harry	60000
164	John	20000
	Jackson	27000

Not allowed as primary key can't contain a NULL value

MISNOURA

3. Referential Integrity Constraints

- A referential integrity constraint is specified between two tables.
- In the Referential integrity constraints, if a foreign key in Table 1 refers to the Primary Key of Table 2, then every value of the Foreign Key in Table 1 must be null or be available in Table 2.

Example:

(Table 1)

EMP_NAME	NAME	AGE	D_No
1	Jack	20	11
2	Harry	40	24
3	John	27	18
4	Devil	38	13

Foreign key

Not allowed as D_No 18 is not defined as a Primary key of table 2 and In table 1, D_No is a foreign key defined

Relationships

(Table 2)

Primary Key

D_No	D_Location
11	Mumbai
24	Delhi
13	Noida

IMS NOIDA

4. Key constraints

- Keys are the entity set that is used to identify an entity within its entity set uniquely.
- An entity set can have multiple keys, but out of which one key will be the primary key. A primary key can contain a unique and null value in the relational table.

Example:

ID	NAME	SEMENSTER	AGE
1000	Tom	1st	17
1001	Johnson	2nd	24
1002	Leonardo	5th	21
1003	Kate	3rd	19
1002	Morgan	8th	22

Not allowed. Because all row must be unique

SQL SET OPERATIONS

SQL supports few Set operations which can be performed on the table data. These are used to get meaningful results from data stored in the table, under different special conditions.

In this tutorial, we will cover 4 different types of SET operations, along with example:

UNION

UNION ALL

INTERSECT

MINUS

UNION Operation

UNION is used to combine the results of two or more SELECT statements. However it will eliminate duplicate rows from its resultset. In case of union, number of columns and datatype must be same in both the tables, on which UNION operation is being applied.

Example of UNION

The **First** table,

ID	Name
1	abhi
2	adam

The **Second** table,

ID	Name
2	adam

IMS NOIDA

3	Chester

Union SQL query will be,

```
SELECT * FROM First

UNION

SELECT * FROM Second;
```

The resultset table will look like,

ID	NAME
1	abhi
2	adam
3	Chester

IMS NOIDA

UNION ALL

This operation is similar to Union. But it also shows the duplicate rows.

Example of Union All

The **First** table,

ID	NAME
1	abhi

2	adam

The **Second** table,

ID	NAME
2	adam
3	Chester

Union All query will be like,

```
SELECT * FROM First

UNION ALL

SELECT * FROM Second;
```

The resultset table will look like,

ID	NAME
1	abhi
2	adam
2	adam
3	Chester

IMS NOIDA

INTERSECT

Intersect operation is used to combine two `SELECT` statements, but it only retuns the records which are common from both `SELECT` statements. In case of **Intersect** the number of columns and datatype must be same.

NOTE: MySQL does not support INTERSECT operator.

Example of Intersect

The **First** table,

ID	NAME
1	abhi
2	adam

The **Second** table,

ID	NAME
2	adam
3	Chester

Intersect query will be,

```
SELECT * FROM First

INTERSECT

SELECT * FROM Second;
```

The resultset table will look like

IMS NOIDA

ID	NAME
2	adam

MINUS

The Minus operation combines results of two `SELECT` statements and return only those in the final result, which belongs to the first set of the result.

Example of Minus

The **First** table,

ID	NAME
1	abhi
2	adam

The **Second** table,

ID	NAME
2	adam
3	Chester

Minus query will be,

```
SELECT * FROM First
```

IMS NOIDA

```
MINUS

SELECT * FROM Second;
```

The resultset table will look like,

ID	NAME
1	abhi

IMS NOIDA

S Q L C o m m a n d s

SQL COMMANDS

- **What is SQL?**

IMS NOIDA

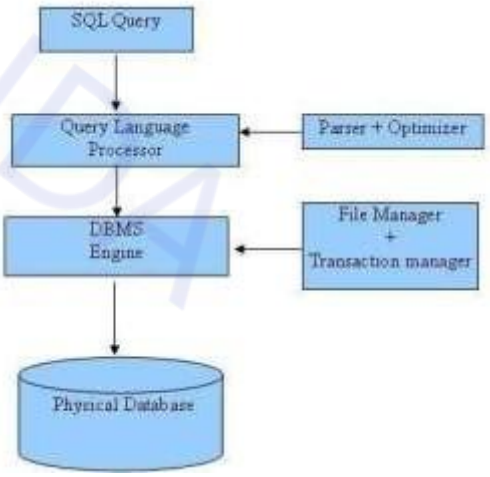
- **Structured Query Language** and it helps to make practice on SQL commands which provides immediate results.
- SQL is Structured Query Language, which is a computer language for storing, manipulating and retrieving data stored in relational database.
- SQL is the standard language for Relation Database System.
- All relational database management systems like MySQL, MS Access, and Oracle, Sybase, Informix, and SQL Server use SQL as standard database language.

➢ Why SQL?

- Allows users to create and drop databases and tables.
- Allows users to describe the data.
- Allows users to define the data in database and manipulate that data.
- Allows users to access data in relational database management systems.
- Allows embedding within other languages using SQL modules, libraries & pre-compilers.
  Allows users to set permissions on tables, procedures, and views

➢ SQL Architecture:

- When you are executing an SQL command for any RDBMS, the system determines the best way to carry out your request and SQL engine figures out how to interpret the task.
- There are various components included in the process.
- These components are:
  - Query Dispatcher ○ Optimization Engines ○ Classic Query Engine ○ SQL Query Engine, etc.
- Classic query engine handles all non-SQL queries but SQL query engine won't handle logical files.
- Simple diagram showing SQL Architecture:



➢ SQL Commands:

- The standard SQL commands to interact with relational databases are CREATE, SELECT, INSERT, UPDATE, DELETE and DROP.
- These commands can be classified into groups based on their nature:

➢ DDL - Data Definition Language:

- ***DDL defines the conceptual schema providing a link between the logical and the physical structure of the database.***

- The functions of the Data Definition Language (DDL) are:
  1. DDL defines the physical characteristics of each record, filed in the record, field‟s data type, field‟s length, field‟s logical name and also specify relationship among those records.
  2. DDL describes the schema and subschema.
  3. DDL indicate the keys of records.
  4. DDL provides data security measures.
  5. DDL provides for the logical and physical data independence.
- Few of the basic commands for DDL are:

Command	Description
CREATE	Creates a new table, a view of a table, or other object in database
ALTER	Modifies an existing database object, such as a table.
DROP	Deletes an entire table, a view of a table or other object in the database.

➢ DML - Data Manipulation Language:

- ***DML provides the data manipulation techniques like selection, insertion, deletion, updation, modification, replacement, retrieval, sorting and display of data or records.***
- DML facilitates use of relationship between the records.
- DML provides for independence of programming languages by supporting several high-level programming languages like COBOL, PL/1 and C++.
- Few of the basic commands for DML are:

Command	Description
SELECT	Retrieves certain records from one or more tables
INSERT	Creates a record
UPDATE	Modifies records
DELETE	Deletes records

IMS NOIDA

➢ DCL - Data Control Language:

- These SQL commands are used for providing security to database objects.
- The different DCL commands are:

Command	Description
GRANT	Gives a privilege to user
REVOKE	Takes back privileges granted from user

➢ TCL – Transaction Control Language:

- It includes commands to control the transactions in a database system.
- The commonly used commands are:

Command	Description
COMMIT	Make all the changes made by the statements issued permanent.
ROLLBACK	Undoes all changes since the beginning of transaction or since a save point.

➢ Data Types in SQL:

The following are the most common data types of SQL:

SL No	DATA TYPE	DESCRIPTION
1	NUMBER	A variable-length column. Allowed values are zero, positive and negative numbers
2	CHAR	A variable length field up to 255 character in length
3	VARCHAR/VARCHAR	A variable length field up to 2000 character in length

IMS NOIDA

	2	
4	DATE/TIME	A fixed length field. The time is stored as a part of the date. The default format is DD/MON/YY
5	LONG	A variable length filed up to 2 GB in length
6	RAW	A variable length filed used for binary data up to 2000 in length
7	LONG RAW	A variable length filed used for binary data up to 2GB in length

1. **NUMBER:**

- Used to store a numeric value in a field column.
- It may be decimal, integer or real value.

General syntax: NUMBER(n, d)

Where **n** specifies the number of digits and **d** specifies the number of digits to right of the decimal point.

Example: marks NUMBER(3), average NUMBER(2, 3)

2. CHAR:

- Used to store a character type data in a column.
- **General syntax:** **CHAR(size)** o Where size represents the maximum (255 Characters) number of characters in a column.
- **Example**: name CHAR(15)

3. VARCHAR/VARCHAR2:

- It is used to store variable length alphanumeric data.

General syntax: VARCHAR(size) / VARCHAR2(size)

- Where size represents the maximum (2000 Characters) number of characters in a column.
- **Example**: address VARCHAR2(50)

4. DATE:

- It is used to store date in columns.
- SQL supports the various date formats other than the standard D-MON-YY.
- **Example**: dob DATE

IMS NOIDA

5. TIME:

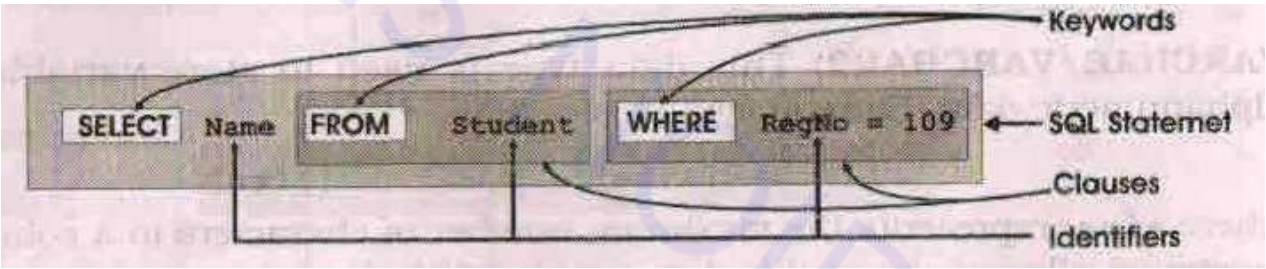
- It is used to store time in columns.
- SQL supports the various time formats other than the standard hh-mm-ss.
- Every DATE and TIME can be added, subtracted or compared as it can be done with other data types.

6. LONG:

1. It is used to store variable length strings of up to 2GB size.
2. **Example**: description LONG

➢ Structure of SQL command:

- Any SQL command is a combination of keywords, identifiers and clauses.
- Every SQL command begins with a keyword (CREATE, SELECT, DELETE and so on) which as a specific meaning to the language.



- SELECT, FROM and WHERE are keywords.
- The clauses are “FROM student” and “WHERE RegNo=109”.
- Here SELECT and FROM are mandatory, but WHERE is optional.
- Name, Student, RegNo, are identifier that refers to objects in the database.
- Name and RegNo are column names, while Student is a table name.
- The equal sign is an operator and 109 is a numeric constant.

➢ What is an Operator in SQL?

- An operator is a reserved word or a character used primarily in an SQL statement's WHERE clause to perform operation(s), such as comparisons and arithmetic operations.
- Operators are used to specify conditions in an SQL statement and to serve as conjunctions for multiple conditions in a statement. ○ Arithmetic operators (+, -, *, / %) ○ Comparison operators (>, <, >=, <=, =, !=, <>, !<, !>)
- Logical operators (AND, OR, NOT, IN, BETWEEN, EXISTS, ALL, ANY, LIKE, UNIQUE)

➢ SQL Logical Operators:

Here is a list of all the logical operators available in SQL.

Operator	Descripti on
ALL	The ALL operator is used to compare a value to all values in another value set.
AND	The AND operator allows the existence of multiple conditions in an SQL statement's WHERE clause.
ANY	The ANY operator is used to compare a value to any applicable value in the list according to the condition.
BETWEE N	The BETWEEN operator is used to search for values that are within a set of values, given the minimum value and the maximum value.
EXISTS	The EXISTS operator is used to search for the presence of a row in a specified table that meets certain criteria.
IN	The IN operator is used to compare a value to a list of literal values that have been specified.
LIKE	The LIKE operator is used to compare a value to similar values using wildcard operators.
NOT	The NOT operator reverses the meaning of the logical operator with which it is used. Eg: NOT EXISTS, NOT BETWEEN, NOT IN, etc. **This is a negate operator.**
OR	The OR operator is used to combine multiple conditions in an SQL statement's WHERE clause.
IS NULL	The NULL operator is used to compare a value with a NULL value.
UNIQUE	The UNIQUE operator searches every row of a specified table for uniqueness (no duplicates).

➢ Implementation of SQL Commands

□ CREATE TABLE

- The SQL **CREATE TABLE** statement is used to create a new table.
- Creating a basic table involves naming the table and defining its columns and each column's data type.
- **Syntax**: Basic syntax of **CREATE TABLE** statement is as follows:

IMS NOIDA

- Here CREATE TABLE is the keyword followed by the Table_name, followed by an open parenthesis, followed by the column names and data types for that column, and followed by a closed parenthesis.
- For each column, a name and a data type must be specified and the column name must be a unique within the table definition.
- Column definitions are separated by commas (,).
- Uppercase and lowercase letters makes no difference in column names.
- Each table must have at least one column. ○ SQL commands should end with a semicolon (;).
- Example: Create a table "STUDENT" that contains five columns: RegNo, Name, Combination, DOB and Fees.

```
CREATE TABLE STUDENT
(
        RegNo           NUMBER (6),
        Name            VARCHAR2 (15),
        Combination     CHAR (4),
        DOB             DATE,
        Fees            NUMBER (9, 2),
        PRIMARY KEY ( RegNo )
);
```

- It creates an empty STUDENT table which looks like this:

RegNo	Name	Combination	DOB	Fees

- Viewing the table information:

```
SQL> DESCRIBE STUDENT;
 Name                                      Null?    Type
 ----------------------------------------- -------- ---------------
 REGNO                                     NOT NULL NUMBER(6)
 NAME                                               VARCHAR2(15)
 COMBINATION                                        CHAR(4)
 DOB                                                DATE
 FEES                                               NUMBER(4,2)
```

eir data type and size along with the constraints.

ALTER Statement:

- The table can be modified or changed by using the ALTER command.
- Syntax: Basic syntax of ALTER TABLE statement is as follows:

IMS NOIDA

```
SQL> ALTER TABLE STUDENT ADD (Address VARCHAR2 (30));
Table altered.
SQL> ALTER TABLE STUDENT MODIFY (Address VARCHAR2 (40));
Table altered.
SQL> ALTER TABLE STUDENT DROP (ADDRESS);
Table altered.
```

Example:

- Using the ALTER TABLE command the following tasks cannot be performed
  - Changing a table name.
  - Changing the column name.
  - Decreasing the size of a column if table data exists.
  - Changing a column's data type.

DROP TABLE:

- The SQL **DROP TABLE** statement is used to remove a table definition and all data, indexes, triggers, constraints, and permission specifications for that table.

- **DROP TABLE** statement is as follows:

```
SQL> DROP TABLE STUDENT;
Table dropped.
```

- Example:

INSERT:

- The SQL **INSERT INTO** Statement is used to add new rows of data to a table in the database.
- Syntax:
- There are two basic syntaxes of INSERT INTO statement as follows:

- Here, column1, column2,...columnN are the names of the columns in the table into which you want to insert data.

- You may not need to specify the column(s) name in the SQL query if you are adding values for all the columns of the table. But make sure the order of the values is in the same order as the columns in the table.

- **METHOD 1:** The SQL INSERT INTO syntax would be as follows:

IMS NOIDA

- Example: Following statements would create six records in STUDENT table:

UPDATE:

- SQL provides the ability to change data through UPDATE command.
- The UPDATE command used to modify or update an already existing row or rows of a table.
- The basic syntax of UPDATE command is given below.

SELECT:

- SQL **SELECT** statement is used to fetch the data from a database table which returns data in the form of result table. These result tables are called result-sets.
- Syntax: The basic syntax of SELECT statement is as follows:

SELECT column1, column2, columnN FROM Table_name;	**Compulsory Part**
[WHERE condition(s)] [GROUPBY column-list] [HAVING condition(s)] [ORDER BY column-name(s)];	Optional Part

- Here, column1, column2...are the fields of a table whose values you want to fetch. If you want to fetch all the fields available in the field, then you can use the following syntax:

```
SQL> SELECT * FROM STUDENT;

     REGNO NAME            COMB DOB             FEES
---------- --------------- ---- --------- ----------
      1401 RAMESH          PCMC 07-AUG-99      14000
      1402 JOHN            PCMB 15-SEP-99      13500
      1403 GANESH          PCME 19-AUG-99      16000
      1404 MAHESH          PCMC 14-JAN-98      17650
      1405 SURESH          PCMB 03-MAR-98      11500
      1410 ARUN            PCMC 01-APR-04      13000
      1411 SHREYA          CEBA                24000

7 rows selected.
```

MS-NOUDA